1.1 Final table list used for analytics and Power BI

1. dim_customer_polars
2. fact_application_polars
3. fact_bureau_polars
4. fact_previous_application_polars
5. fact_installments_polars
6. fact_pos_cash_polars
7. fact_credit_card_polars
8. fact_model_scores_polars
9. dim_risk_segment_polars
10. kpi_snapshot_polars

1.2 Row counts and role

1. dim_customer_polars: 356,255 rows. Customer profile dimension.
2. fact_application_polars: 356,255 rows. Core loan application fact.
3. fact_bureau_polars: 305,811 rows. Bureau aggregates per customer.
4. fact_previous_application_polars: 338,857 rows. Previous application aggregates.
5. fact_installments_polars: 339,587 rows. Installment behavior aggregates.
6. fact_pos_cash_polars: 337,252 rows. POS cash performance aggregates.
7. fact_credit_card_polars: 103,558 rows. Credit card behavior aggregates.
8. fact_model_scores_polars: 356,255 rows. ML prediction outputs.
9. dim_risk_segment_polars: 5 rows. Risk band rollup.
10. kpi_snapshot_polars: 1 row. Executive KPI snapshot.

1.3 Attribute dictionary: dim_customer_polars

1. SK_ID_CURR (Int64): Customer unique key. Primary key used to link all facts.
2. CODE_GENDER (String): Customer gender category.
3. NAME_EDUCATION_TYPE (String): Highest known education band.
4. NAME_FAMILY_STATUS (String): Family status (single, married, etc.).
5. NAME_INCOME_TYPE (String): Income source type (working, pensioner, etc.).
6. NAME_HOUSING_TYPE (String): Housing category (house, rented, etc.).
7. REGION_RATING_CLIENT (Int64): Region score assigned in source system.
8. CNT_CHILDREN (Int64): Number of children.
9. CNT_FAM_MEMBERS (Float64): Number of family members.
10. DAYS_BIRTH (Int64): Days before application date (negative in source).
11. DAYS_EMPLOYED (Int64): Days employed before application date (negative in source).
12. AMT_INCOME_TOTAL (Float64): Total declared income.
13. AGE_YEARS (Float64): Engineered age in years from DAYS_BIRTH.
14. EMPLOYMENT_YEARS (Float64): Engineered employment duration in years.

Why this table exists:

1. It keeps profile attributes in one place for slicing/filtering.
2. It avoids repeating profile fields across all fact tables.

1.4 Attribute dictionary: fact_application_polars

1. SK_ID_CURR (Int64): Customer key, links to dim_customer.
2. DATASET (String): train or test marker.
3. TARGET (Int64): Default label where available.
4. AMT_CREDIT (Float64): Requested credit amount.
5. AMT_ANNUITY (Float64): Annuity/periodic payment amount.
6. AMT_GOODS_PRICE (Float64): Goods price tied to application.
7. AMT_INCOME_TOTAL (Float64): Income repeated for fact-level calculations.
8. NAME_CONTRACT_TYPE (String): Contract type.
9. FLAG_OWN_CAR (String): Car ownership flag.
10. FLAG_OWN_REALTY (String): Real estate ownership flag.
11. RISK_SCORE_BASE (Float64): Base risk score engineered from external risk sources.
12. RISK_BAND (String): Categorical risk segment from base risk score.

Why this table exists:

1. It is the core business fact for loan/default analysis.

2. It combines financial amount fields with base risk segmentation.

1.5 Attribute dictionary: fact_bureau_polars

1. SK_ID_CURR (Int64): Customer key.
2. BUREAU_LOAN_COUNT (UInt32): Total bureau loans.
3. BUREAU_ACTIVE_COUNT (UInt32): Count of active bureau loans.
4. BUREAU_CLOSED_COUNT (UInt32): Count of closed bureau loans.
5. BUREAU_DEBT_SUM (Float64): Sum of bureau debt amount.
6. BUREAU_OVERDUE_SUM (Float64): Sum of overdue amount in bureau.
7. BUREAU_CREDIT_SUM (Float64): Sum of total credit amount in bureau.

Why this table exists:

1. External credit behavior is usually predictive for default risk.
2. This aggregation reduces many-to-one source complexity.

1.6 Attribute dictionary: fact_previous_application_polars

1. SK_ID_CURR (Int64): Customer key.
2. PREV_APP_COUNT (UInt32): Number of previous applications.
3. PREV_APPROVED_COUNT (UInt32): Number of approved previous applications.
4. PREV_REFUSED_COUNT (UInt32): Number of refused previous applications.
5. PREV_AMT_APPLICATION_SUM (Float64): Sum of previously requested amounts.
6. PREV_AMT_CREDIT_SUM (Float64): Sum of previously granted credit amounts.

Why this table exists:

1. Previous decisions provide behavioral context.
2. Approval/refusal balance helps detect risk patterns.

1.7 Attribute dictionary: fact_installments_polars

1. SK_ID_CURR (Int64): Customer key.
2. INSTAL_COUNT (UInt32): Number of installment records.
3. INST_DAYS_LATE_MEAN (Float64): Mean delay days vs scheduled payment.
4. INST_DAYS_LATE_MAX (Float64): Maximum delay days.
5. INST_LATE_RATE (Float64): Fraction of late installments.
6. INST_PAYMENT_SUM (Float64): Total installment payments.

Why this table exists:

1. Late payment behavior is one of the strongest risk signals.
2. Mean/max lateness provides both trend and worst-case views.

1.8 Attribute dictionary: fact_pos_cash_polars

1. SK_ID_CURR (Int64): Customer key.
2. POS_RECORD_COUNT (UInt32): Number of POS records.
3. POS_DPD_MEAN (Float64): Mean days past due.
4. POS_DPD_MAX (Int64): Maximum days past due.
5. POS_DPD_DEF_MEAN (Float64): Mean severe delinquency metric.
6. POS_DPD_DEF_MAX (Int64): Maximum severe delinquency metric.

Why this table exists:

1. POS delinquency is useful for short-term repayment stress.
2. Complements installment and bureau behavior.

1.9 Attribute dictionary: fact_credit_card_polars

1. SK_ID_CURR (Int64): Customer key.
2. CC_RECORD_COUNT (UInt32): Number of credit card records.
3. CC_BALANCE_SUM (Float64): Total card balance amount.
4. CC_DRAWINGS_SUM (Float64): Total card drawing amount.
5. CC_PAYMENT_SUM (Float64): Total card payment amount.
6. CC_UTILIZATION_MEAN (Float64): Mean utilization ratio.
7. CC_UTILIZATION_MAX (Float64): Max utilization ratio.

Why this table exists:

1. Utilization ratio often reflects revolving-credit stress.
2. Drawings vs payments indicates repayment discipline.

1.10 Attribute dictionary: fact_model_scores_polars

1. SK_ID_CURR (Int64): Customer key.
2. DATASET (String): train or test.
3. TARGET (Float64): Label where available.
4. PRED_LABEL (Int32): Predicted default class.
5. PRED_SCORE (Float64): Predicted default risk score/probability.
6. PRED_RISK_BAND (Categorical): Risk band derived from score quantiles.

Why this table exists:

1. Connects ML output to business dashboarding.
2. Enables operational prioritization by risk band.

1.11 Attribute dictionary: dim_risk_segment_polars

1. RISK_BAND (String): Very Low, Low, Medium, High, Very High.
2. CUSTOMER_COUNT (UInt32): Number of customers in each band.
3. DEFAULT_COUNT (Int64): Default count per band.
4. AVG_CREDIT (Float64): Average credit amount per band.
5. AVG_INCOME (Float64): Average income per band.
6. DEFAULT_RATE (Float64): Defaults divided by customer count.

Why this table exists:

1. Gives management-ready segmentation view.
2. Makes high-level risk storytelling simple.

1.12 Attribute dictionary: kpi_snapshot_polars

1. TOTAL_CUSTOMERS (Int64): Distinct customers in model scope.
2. TRAIN_CUSTOMERS (Int64): Customers from training dataset.
3. TEST_CUSTOMERS (Int64): Customers in scoring-only dataset.
4. DEFAULT_COUNT (Int64): Count of target=1 in train scope.
5. DEFAULT_RATE (Float64): Mean target in train scope.
6. TOTAL_CREDIT_AMOUNT (Float64): Total requested credit.
7. AVG_CREDIT_AMOUNT (Float64): Mean requested credit.
8. AVG_ANNUITY (Float64): Mean annuity amount.
9. AVG_INCOME (Float64): Mean income amount.

Why this table exists:

1. Provides instant executive summary numbers.
2. Avoids recalculating heavy KPIs repeatedly.

---

2. Data Loading, Cleaning, and Revised Schema Design

2.1 Data loading process

1. Raw source files are read from data/raw.

2. If parquet cache exists in data/processed/parquet_cache, notebook reads parquet first for speed.
3. Processed train/test feature files are read from data/processed for modeling.
4. Datamart notebook merges application_train and application_test using a DATASET flag.

Why this matters:

1. Parquet is faster and more memory efficient than CSV for repeated runs.
2. Keeping train/test marker allows one unified analytic model while preserving evaluation logic.

2.2 Data cleaning steps applied

1. Type normalization for key fields like TARGET.
2. Null-safe calculations for ratios via safe_divide.
3. Handling missing EXT_SOURCE values through median fallback before risk score construction.
4. Aggregate-heavy tables are grouped by SK_ID_CURR to produce one row per customer in each behavior fact.
5. In modeling flow, pd.NA is converted to np.nan to avoid sklearn boolean ambiguity issues.

Why these steps are required:

1. Inconsistent types and null patterns can break model pipelines.
2. Ratio calculations without null-safe logic can produce invalid values.
3. BI semantic models work best with stable one-grain fact tables.

2.3 Revised schema design (star-like model)

Business grain and join key:

1. Main key: SK_ID_CURR.
2. One profile dimension table.
3. Multiple behavior fact tables at customer level.
4. Model score fact aligned to same customer key.

Recommended relationship layout:

1. dim_customer_polars is central dimension.
2. fact_application_polars and fact_model_scores_polars attach directly to dim_customer_polars.
3. Other behavior facts also attach by SK_ID_CURR.
4. dim_risk_segment_polars can be used as helper dimension for risk views.

Why this schema is useful:

1. Easy for beginners to understand and filter.
2. Supports both diagnostic (model) and business (portfolio) analytics.

3. Limits many-to-many ambiguity common in raw Home Credit files.

---

3. Feature Engineering: What Was Done and Why

3.1 Core engineered fields in datamart

1. EXT_SOURCE_MEAN:
1. Mean of EXT_SOURCE_1, EXT_SOURCE_2, EXT_SOURCE_3.
2. Why: combines multiple external risk proxies into one stable signal.

2. RISK_SCORE_BASE:
1. Computed as 1 - EXT_SOURCE_MEAN with median fallback and clipped between 0 and 1.
2. Why: converts higher external score into lower risk and vice versa, creating intuitive risk scale.

3. RISK_BAND:
1. Quantile-based segmentation of RISK_SCORE_BASE into five bands.
2. Why: business users prefer categories over raw continuous scores.

4. AGE_YEARS:
1. Derived from DAYS_BIRTH.
2. Why: years are easier to read than negative days.

5. EMPLOYMENT_YEARS:
1. Derived from DAYS_EMPLOYED with sign correction and null-safe logic.
2. Why: helps profile stability and income reliability context.

6. DAYS_LATE and IS_LATE:
1. Derived in installments from payment day minus scheduled day.
2. Why: direct repayment behavior signal for risk.

7. CC_UTILIZATION:
1. AMT_BALANCE / AMT_CREDIT_LIMIT_ACTUAL, null-safe.
2. Why: high utilization often indicates financial stress.

3.2 Aggregation engineering

From granular transactional tables to customer-level features:

1. Counts: total records, active/closed counts, approved/refused counts.
2. Sums: debt, overdue, payment, drawings, application amounts.
3. Means: average lateness, average utilization, average DPD.

4. Max values: worst delinquency/utilization events.

Why this approach:

1. Model and BI both need one customer row per feature family.
2. Aggregates preserve behavior signal while reducing data volume.

3.3 Prediction engineering

1. PRED_SCORE from model inference.
2. PRED_LABEL from thresholded class output.
3. PRED_RISK_BAND from score quantiles.

Why this helps business teams:

1. Numeric score supports ranking.
2. Label supports binary action.
3. Risk band supports dashboard communication and prioritization.

---

4. Problem Statement: What We Are Solving and How

4.1 Business problem

Home Credit wants to identify who is likely to default so that lending decisions, portfolio monitoring, and risk controls improve.

4.2 Analytical objective

1. Build a robust classification model for default risk.
2. Create interpretable, segment-level insights for business users.
3. Deliver outputs to Power BI for continuous monitoring.

4.3 Why this is challenging

1. Data is large and spread across many tables.
2. Classes are imbalanced (default is minority class).
3. High accuracy can be misleading when minority recall is low.

4.4 Solution strategy

1. Build fast repeatable data layer using parquet and Polars.

2. Engineer behavior and profile signals at customer grain.
3. Train and compare multiple models via PyCaret.
4. Focus on AUC and risk ranking quality, not only accuracy.
5. Export model scores and BI facts for decision dashboards.

---

5. PyCaret Workflow: All Steps, What We Did, and Why

5.1 Data preparation before setup

1. Load processed train/test features.
2. Use sample strategy for runtime control.
3. Convert Polars outputs to pandas before PyCaret.
4. Replace pd.NA with np.nan.

Why:

1. PyCaret/sklearn requires stable pandas-compatible null semantics.
2. Sampling accelerates model iteration.

5.2 setup step

Parameters used in latest stronger run:

1. fold = 3
2. preprocess = True
3. simple median/most_frequent imputation
4. n_jobs = -1
5. fix_imbalance = False (runtime trade-off)
6. remove_multicollinearity = False (runtime trade-off)

Why:

1. fold=3 gives more stable validation than fold=2.
2. preprocessing and imputation provide baseline pipeline reliability.
3. disabling expensive transforms keeps runtime practical.

5.3 compare_models step

1. Compared curated model IDs: lightgbm, rf, et, gbc, ada, lr, ridge.
2. Used sort='AUC' and selected top N models.

Why:

1. Curated list balances speed and quality.
2. AUC is more meaningful than accuracy for imbalanced problems.

5.4 tune_model step

1. Controlled tuning only for top model.
2. Low n_iter for speed.

Why:

1. Full tuning for many models is the major bottleneck.
2. Top-model-only tuning captures most gains with lower cost.

5.5 candidate evaluation and selection

1. predict_model called per candidate on holdout split.
2. Holdout metrics captured and concatenated.
3. Best model chosen by highest holdout AUC.

Why:

1. Prevents over-reliance on CV summary alone.
2. Gives transparent head-to-head comparison.

5.6 finalize and score

1. finalize_model to retrain best pipeline.
2. save_model for reproducibility.
3. score train and test and export structured prediction outputs.

Why:

1. Finalized model is production-style artifact.
2. Exported scores directly power business prioritization and dashboards.

5.7 Current measured outcomes

From latest holdout file:

1. Best AUC: 0.7495 (Gradient Boosting).
2. Best Accuracy: 0.9171.

Interpretation for beginners:

1. Accuracy is high due to class imbalance.

2. AUC tells us ranking quality is improved.

3. Recall is still modest and can be improved with threshold strategy and imbalance treatment in future iterations.

---

6. Calculated Columns and Measures: What, Formula Logic, and Why

This section includes both Python-calculated fields and recommended Power BI measures.

6.1 Calculated columns created in Python layer

1. EXT_SOURCE_MEAN:

1. Mean of EXT_SOURCE_1, EXT_SOURCE_2, EXT_SOURCE_3.

2. Why: combined external score proxy.

2. RISK_SCORE_BASE:

1. 1 - EXT_SOURCE_MEAN, null-safe and clipped.

2. Why: intuitive higher=more-risk scale.

3. RISK_BAND:

1. Quantile bucket from RISK_SCORE_BASE.

2. Why: boardroom-friendly segmentation.

4. AGE_YEARS:

1. (-DAYS_BIRTH)/365.

2. Why: human-readable age.

5. EMPLOYMENT_YEARS:

1. if DAYS_EMPLOYED<0 then (-DAYS_EMPLOYED)/365.

2. Why: employment tenure insight.

6. DAYS_LATE:

1. DAYS_ENTRY_PAYMENT - DAYS_INSTALMENT.

2. Why: absolute lateness behavior.

7. IS_LATE:

1. DAYS_LATE > 0 cast to 0/1.

2. Why: easy late-rate aggregation.

8. CC_UTILIZATION:

1. AMT_BALANCE / AMT_CREDIT_LIMIT_ACTUAL (null-safe).

2. Why: revolving pressure indicator.

9. PRED_RISK_BAND:

1. Quantile bucket on PRED_SCORE.

2. Why: operational targeting by risk bucket.

6.2 Recommended Power BI measures (DAX)

Use these in model report for consistent KPI logic.

1. Total Customers

1. DAX: DISTINCTCOUNT(fact_application_polars[SK_ID_CURR])

2. Why: denominator for most ratios.

2. Default Count

1. DAX: SUM(fact_application_polars[TARGET])

2. Why: numerator for observed risk.

3. Default Rate

1. DAX: DIVIDE([Default Count], [Total Customers])

2. Why: baseline observed portfolio risk.

4. Avg Credit Amount

1. DAX: AVERAGE(fact_application_polars[AMT_CREDIT])

2. Why: monitor loan ticket size.

5. Total Credit Amount

1. DAX: SUM(fact_application_polars[AMT_CREDIT])

2. Why: exposure magnitude.

6. Avg Income

1. DAX: AVERAGE(fact_application_polars[AMT_INCOME_TOTAL])

2. Why: economic profile context.

7. Avg Pred Score

1. DAX: AVERAGE(fact_model_scores_polars[PRED_SCORE])

2. Why: model-implied risk level.

8. Predicted High Risk Customers

1. DAX: CALCULATE([Total Customers], fact_model_scores_polars[PRED_RISK_BAND] IN {"High","Very High"})

2. Why: prioritize review queue.

9. Predicted High Risk Share

1. DAX: DIVIDE([Predicted High Risk Customers], [Total Customers])

2. Why: portfolio stress indicator.

10. Bureau Overdue per Customer

1. DAX: DIVIDE(SUM(fact_bureau_polars[BUREAU_OVERDUE_SUM]), [Total Customers])

2. Why: average overdue intensity.

11. Late Installment Rate (weighted)

1. DAX: DIVIDE(SUMX(fact_installments_polars, fact_installments_polars[INST_LATE_RATE] * fact_installments_polars[INSTAL_COUNT]), SUM(fact_installments_polars[INSTAL_COUNT]))

2. Why: more reliable than simple unweighted average.

12. Utilization Mean

1. DAX: AVERAGE(fact_credit_card_polars[CC_UTILIZATION_MEAN])

2. Why: card stress signal.

13. Approval Rate (historical)

1. DAX: DIVIDE(SUM(fact_previous_application_polars[PREV_APPROVED_COUNT]), SUM(fact_previous_application_polars[PREV_APP_COUNT]))

2. Why: lending pipeline quality context.

14. Refusal Rate (historical)

1. DAX: DIVIDE(SUM(fact_previous_application_polars[PREV_REFUSED_COUNT]), SUM(fact_previous_application_polars[PREV_APP_COUNT]))

2. Why: past rejection intensity.

15. Model AUC (static from metrics table)

1. DAX: MAX(model_metrics[BestAUC]) if table imported.

2. Why: model health indicator on overview page.

---

7. Charts and KPI by Dashboard Page: What and Why

Below is a full page design that a beginner can implement.

7.1 Page 1: Executive Overview

Purpose:

1. Give leadership immediate health snapshot of portfolio and model risk.

KPI cards:

1. Total Customers

2. Default Count

3. Default Rate

4. Total Credit Amount
5. Avg Credit Amount
6. Avg Pred Score
7. Predicted High Risk Share

Charts:

1. Column chart: Default Rate by RISK_BAND.
1. What: compare observed risk across segments.
2. Why: validates segmentation usefulness.

2. Stacked bar: Customer Count by PRED_RISK_BAND and DATASET.
1. What: train/test distribution by model risk.
2. Why: checks score stability between populations.

3. Donut: FLAG_OWN_REALTY share.
1. What: property ownership mix.
2. Why: quick socioeconomic profile signal.

4. Line or area chart: Avg Pred Score by REGION_RATING_CLIENT.
1. What: regional risk trend.
2. Why: geographic risk prioritization.

7.2 Page 2: Customer Segmentation

Purpose:

1. Understand which customer groups contribute most to risk.

KPI cards:

1. Avg Income
2. Avg Credit
3. Default Rate (filtered)
4. High Risk Customers (filtered)

Charts:

1. Matrix: Education x Income Type with Default Rate and Customer Count.
1. What: two-dimensional risk profile.
2. Why: identifies high-risk customer personas.

2. Bar chart: Default Rate by NAME_FAMILY_STATUS.
1. What: family status vs risk.
2. Why: common demographic segmentation axis.

3. Scatter: AMT_INCOME_TOTAL vs AMT_CREDIT, color by PRED_RISK_BAND.

1. What: affordability pattern.

2. Why: reveals clusters of risky credit-income mismatches.

4. Histogram/bin chart: AGE_YEARS distribution by TARGET.

1. What: age-based default tendency.

2. Why: supports policy discussions.

7.3 Page 3: Credit Behavior and Delinquency

Purpose:

1. Diagnose why customers look risky using behavioral facts.

KPI cards:

1. Bureau Overdue per Customer
2. Late Installment Rate
3. Avg CC Utilization
4. Refusal Rate

Charts:

1. Bar chart: BUREAU_ACTIVE_COUNT vs DEFAULT_RATE.

1. What: active external credit load impact.

2. Why: debt burden relationship.

2. Bar chart: INST_DAYS_LATE_MAX bands vs Default Rate.

1. What: maximum lateness impact.

2. Why: severe delinquency early warning.

3. Combo chart: PREV_APPROVED_COUNT and PREV_REFUSED_COUNT by segment.

1. What: historical decision dynamics.

2. Why: prior application behavior context.

4. Scatter: CC_UTILIZATION_MEAN vs INST_LATE_RATE.

1. What: revolving usage vs installment discipline.

2. Why: identifies financially stressed clusters.

7.4 Page 4: Model Monitoring and Actioning

Purpose:

1. Connect model outputs with business actions.

KPI cards:

1. Best Holdout AUC
2. Best Holdout Accuracy
3. Predicted High Risk Customers
4. Predicted High Risk Share

Charts:

1. Distribution chart: PRED_SCORE histogram.
1. What: spread of model confidence.
2. Why: helps choose operational thresholds.

2. Bar chart: PRED_RISK_BAND vs observed DEFAULT_RATE (train rows only).
1. What: calibration by risk bucket.
2. Why: checks whether higher score means higher observed default.

3. Table: Top high-risk customers (SK_ID_CURR, score, key behavior metrics).
1. What: action list for review team.
2. Why: directly operational output.

4. Scenario slicer and card:
1. What: threshold parameter and impacted customer count.
2. Why: decision simulation for risk policy.

---

8. What We Have Solved So Far

1. Built an end-to-end repeatable pipeline from raw files to BI-ready datamart.
2. Trained and evaluated multiple models with PyCaret.
3. Improved model ranking quality to holdout AUC 0.7495.
4. Generated production-style prediction outputs and risk segmentation.
5. Generated multiple business-ready fact tables for Power BI dashboarding.

---

9. New Analyst FAQ (Simple Language)

1. Why is accuracy high but recall low?
1. Because most customers are non-default, so predicting many zeros can still be "accurate".

2. Why care about AUC?
1. AUC measures ranking quality across thresholds, very useful for imbalanced credit risk.

3. Why both RISK_BAND and PRED_RISK_BAND?

1. RISK_BAND comes from base external features.
2. PRED_RISK_BAND comes from model output score.
3. Comparing both gives business and model perspective.

4. Why many fact tables instead of one giant table?

1. Better performance and cleaner semantics in Power BI.
2. Easier to maintain and explain.

5. Can we compute true test accuracy?

1. No, application_test has no labels. Use holdout metrics and leaderboard/CV metrics.

---

10. Practical Next Steps

1. Import all *_polars parquet tables into Power BI.
2. Build relationships on SK_ID_CURR.
3. Create DAX measures from Section 6.2.
4. Build pages from Section 7.
5. Validate filters and totals across pages.
6. Share dashboard with business stakeholders for threshold and action policy decisions.